INF1103 Week 8 Lab 6 Submission

1. UML Class Diagram Specification

Product Class (`product.py`)

Attributes:

- `_product_id`: String
- `_name`: String
- `_price`: Float
- `_stock`: Integer

Methods:

- `__init__(product_id, name, price, stock)`: Initializes product attributes.
- `name`: Property getter for accessing product name.

`display_information()`: Prints formatted details of the product.

Inventory Class (`inventory.py`)

Attributes:

- `products`: List of `Product` instances.

Methods:

- `__init__()`: Initializes an empty product list.
- `add_product(product)`: Appends a `Product` object to inventory.
- `display_all()`: Iterates through `products` and calls `display_information()`.

`search_product(search_name)`: Searches `products` matching `search_name`.

Diagram Structure

```
None
+---------------------------------------+
+---------------------------------------+
|               Product                 |              |
Inventory               |
+---------------------------------------+
+---------------------------------------+
| - _product_id: str                    |              | - products: list
|
| - _name: str                          |
+---------------------------------------+
| - _price: float                       |<>----------->| + __init__()
|
| - _stock: int                         |              | + add_product(product)
|
+---------------------------------------+              | + display_all()
|
| + __init__(product_id, name, ... )    |              | +
search_product(search_name)         |
| + name() [Property]                   |
+---------------------------------------+
| + display_information()               |
+---------------------------------------+
```

2. Lab Reflection Answers

1. How does using private attributes (`_product_id`, `_name`, `_price`, `_stock`) enforce encapsulation?

Using an underscore prefix signals that these attributes should not be altered directly outside the class. Instead, access is managed through defined interfaces (like the `@property` getter or `display_information()`). This prevents external functions from messing up internal data states accidentally.

2. How does splitting `Product` and `Inventory` follow the Single Responsibility Principle (SRP)?

Each file handles one specific job:

- **Product** only cares about managing single product data and formatting its output.

- **Inventory** only cares about storing, querying, and managing the list of products.

If we need to change how a single item works, we only touch `product.py`. If we need to update search or collection logic, we only touch `inventory.py`. This keeps the code clean and easy to debug.